

CPT202 Assignment 2

Group Report for Software Engineering Group Project

2023/2024 Semester 2

< Project B: Fitness Trainer Appointment System >

< 10 May 2024>

<URL: <http://120.55.188.177:8080/>>

Group number: <23>

Group Members:

1. <Qibin Gu, 2143132>
2. <Yuchen Gu, 2142038>
3. <Haoran Qin, 2143434>
4. < Zongheng Liu, 2144695 >
5. < Jiarui Chen, 2145016 >
6. < Yinuo Wang, 2143466 >
7. < Jixiao He, 2144984 >
8. < Hanlin Zheng, 2145799 >

Contents

<i>I. Introduction (Suggest 1 pages)</i>	3
1.1 Problem Statement	3
1.2 Aims of The Project	3
1.3 Project Scope	3
1.4 User Characteristics	3
1.5 Assumptions and Dependencies	3
1.6 Project Risks.....	3
<i>II. Architectural Design (Suggest 2.5 pages)</i>	4
2.1 System Architecture	4
2.2 Frontend System	4
2.3 Backend System	4
2.4 High-level Database Design.....	5
<i>III. Software Design (Suggest 2.5 page)</i>	5
3.1 Software Modules	5
3.2 High-level Process Flow	5
3.3 Software Support Services	6
3.4 Coding Structure and Convention.....	6
3.5 Software Configuration and Production Environment.....	7
<i>IV. Software testing (Suggest 1.5 page)</i>	7
<i>V. Conclusion and future work (Suggest 0.5 pages)</i>	8
<i>References (If needed, not count into the limit of 8 pages)</i>	9
<i>Appendix (Not count into the limit of 8 pages)</i>	9

I. Introduction

1.1 Problem Statement

The fitness industry has experienced a significant increase in demand in recent years, driven by a growing emphasis on health and wellness. This trend underscores the pressing need for innovative and user-centric fitness software solutions to meet the evolving needs of both fitness centers and their customers. Current challenges in fitness software platforms include complex user interfaces, inefficient appointment booking processes, complex payment procedures, and inconsistent user experiences. These challenges often result in user frustration, decreased satisfaction, and potential drop-offs in bookings. To address these issues effectively, it is imperative to develop a robust and user-friendly Fitness Appointment Management System. This system should prioritize user-centric design principles, seamless integration with third-party services, personalized guidance, and effective communication tools. By streamlining the appointment process and enhancing client satisfaction, such a system can empower users to achieve their health and fitness goals more effectively.

1.2 Aims of The Project

The primary objective of this project is to design and implement an intuitive web-based platform:

For customers, they can choose to register an account to become a member of the fitness center, and the website will provide members with a series of core services for booking fitness classes.

For managers, it provides a management background for the manager of the fitness center. It provides a series of apis for the management and operation of the fitness center to support the management of member and coach information and the visualization of data for further business development.

1.3 Project Scope

The scope of the project is illustrated in the Figure 1:



Figure 1. the Use Case Diagram of the System

The project scope encompasses the development of a web-based Fitness Appointment Management System. It includes features such as user registration, appointment scheduling, fitness plan purchase, and management functionalities for both customers and managers. The scope also covers integration with third-party services, ensuring data security, and providing a user-friendly interface.

1.4 User Characteristics

The system will cater to four main user types: visitors, members, coaches, and managers.

Visitors have the option to register as members, unlocking access to a range of features.

Members for the customer who have registered, can seamlessly log in to the platform to purchase fitness plan, schedule appointments, review their appointment history, and offer valuable feedback.

Coaches enjoy a suite of functionalities tailored to their role. They can efficiently manage their students' class schedules, stay informed through timely notifications, and assess student progress during sessions.

Managers play a pivotal role in maintaining the system's integrity and effectiveness. This entails overseeing the main database, ensuring its smooth operation, and proactively engaging with members through targeted marketing notifications. Additionally, the manager possesses the authority to review and manage appointments, delve into insightful statistical reports, and facilitate the promotion or demotion of trainers across different ranks as needed.

Together, these user personas contribute to a dynamic and user-centric ecosystem aimed at optimizing the fitness appointment experience for all stakeholders involved.

1.5 Assumptions and Dependencies

The successful implementation of the project is contingent upon certain assumptions and dependencies. These include assumptions regarding the availability of necessary resources such as development tools and technologies, as well as dependencies on external factors such as third-party APIs for payment processing and email notifications.

1.6 Project Risks

While the project presents numerous opportunities for enhancing the functionality and efficiency of the fitness center, it also entails certain risks that must be carefully managed. These risks include technical challenges associated with system integration, potential security vulnerabilities, and the need for robust testing to ensure the reliability and performance of the platform.

II. Architectural Design

2.1 System Architecture

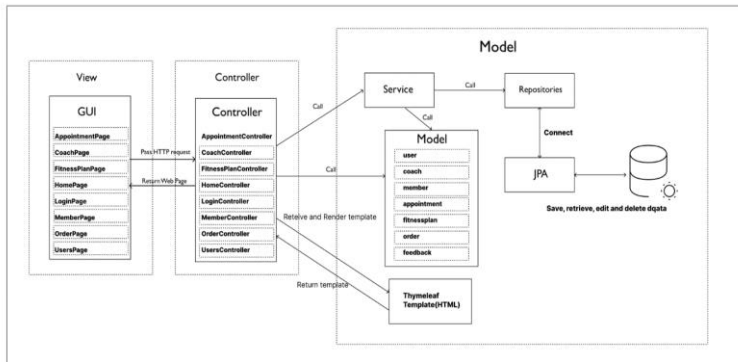


Figure 2. The MVC Architecture of the System

Our group chose to use the MVC (Model View Controller) [1] architecture to develop fitness reservation software, mainly based on its many advantages in software engineering. Firstly, the MVC pattern provides a clear structure that separates different aspects of an application, including data logic (Model), user interface (View), and control logic (Controller) [2]. This separation enables the team to better organize and manage code, improving its maintainability and scalability. Secondly, adopting the MVC pattern helps to achieve code reuse, as each component can be independently developed and tested, thereby reducing the complexity of the development process. In addition, the MVC pattern also facilitates collaborative development among multiple people, as different developers can work on different components simultaneously without interfering with each other. Finally, the MVC pattern separates the user interface of the software from the data logic, making it easier to modify and update the interface without affecting the data processing logic. In summary, we chose MVC as the development architecture for fitness reservation software based on its clear structure, code reusability, multiplayer collaboration, and separation of interface and logic.

Model: In the context of Spring MVC, model classes are utilized to represent the application's data. They often work in conjunction with the view layer to present the data to the user. For example, a model object can store data obtained from a database and be passed to a view for rendering the data on a web page. The model object can also validate user input before passing it to the service layer for further processing.

View: The view is responsible for presenting the application's user interface, displaying data from the model in a visual format such as a webpage or graphical interface. In our project, the view

corresponds to the front-end layer implemented with the Thymeleaf template engine. It handles the presentation of data and user interactions, including UI elements like buttons and input fields.

Controller: The controller serves as the intermediary between the model and the view, managing user inputs and coordinating the interaction between these components. When users interact with the application by clicking buttons or entering information, the controller adjusts the model's state accordingly and ensures the view is updated to reflect these changes.

2.2 Frontend System

In our project, the front-end is implemented using the Thymeleaf template engine, which is a server-side Java template engine that can handle HTML, XML, JavaScript, CSS [3], and even plain text content. Thymeleaf is responsible for handling data representation and user interaction, including UI elements such as buttons and input fields. Through HTTP requests and responses, the view layer utilizes Thymeleaf to present the data retrieved from the model and interact with the user. Our goal is to utilize modern network technologies to create a visually appealing and user-friendly interface. In the fitness reservation system, the View component is responsible for displaying data and processing user interactions, including functions such as user registration, login, reservation, and reservation history. We adopt a responsive design approach to ensure compatibility of the user interface across different devices, screen sizes, and resolutions. In addition, we use front-end frameworks to create reusable UI components and handle dynamic user interactions more effectively. This will improve the overall development speed and bring a smoother user experience.

2.3 Backend System

In term of the backend, the Control layer calls the Service layer interface, and then the Service layers use the Dao layer to perform database CRUD operations [4].

1. **Entity Layer:** The entity module represents a data model implemented by Java classes, corresponding to tables in the database for fitness appointment related information. The entity layer is responsible for defining the structure of data and mapping data when interacting with the database. These entity classes typically represent the core data objects in our fitness reservation system, such as members, reservation records, etc.
2. **DAO (Data Access Object) Layer:** The data access object layer is responsible for interacting with data storage (usually databases), and Mapper is its specific implementation. In this layer, I used

mybatis Plus to achieve communication and data querying between Java and MYSQL databases. It provides a set of interfaces and methods for performing CRUD operations on the database (create, read, update, delete). The DAO layer hides database details and provides an abstract way to access data, making it easier to reuse in other parts of the application.

3. **Service layer:** The service layer contains the business logic of the application. Annotations can be used to inject Mappers to coordinate the interaction of multiple DAO objects. In addition, the service layer can call the interfaces and methods provided by the Mapper layer to perform database creation, reading, updating, and deletion operations.
4. **Controller layer:** The controller layer interprets user actions and forwards them to appropriate services or business components for processing. It is also responsible for passing the results to the view layer to update the user interface. Controllers typically handle multiple requests, each corresponding to a method.

2.4 High-level Database Design

Our database design uses the database management system MySQL to store multiple tables of different types of data to represent the direct relationships between different data. This database contains the Entity Relationship Diagram (ERD) used in the following Figure 3, which displays the overall design of our project's database. The relationships between tables are maintained through foreign key constraints.

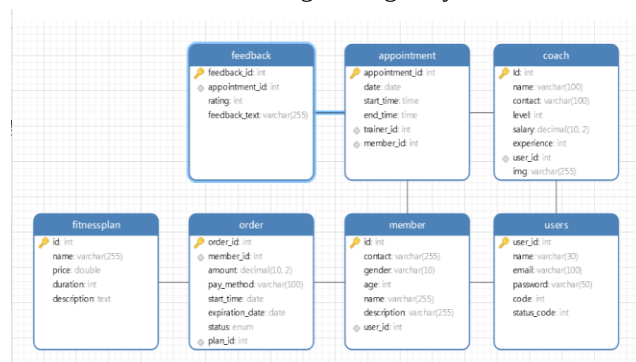


Figure 3. The ERD of Database

Entity Identification: We identified the various entities present in our system as a basis for building tables. Each entity's attributes were carefully selected based on their relevance to the system's functionality and data requirements. The explanation of the structure is:

1. **Users:** The relevant information entity that the user logs into the website
2. **Member:** The underlying information entity for fitness membership

3. **Coach:** Basic information entities for coaches (including work experience and ratings).
4. **Appointment:** Record the member's appointment record entity on the fitness platform.
5. **Fitnessplan:** An entity that records the actual content of the fitness program (including program levels, prices, etc.)
6. **Order:** A record entity for a user's order to purchase a fitness program.
7. **Feedback:** Used to record user feedback

Data Normalization: We applied principles of data normalization (using 3NF) to reduce redundancy and improve data consistency [5]. By decomposing data into appropriate tables and ensuring each table represents a single subject, we minimize data duplication and maintain integrity.

III. Software Design

3.1 Software Modules

We divide the software into four main modules according to the different roles of using the system, which are the tourist (who haven't logged in) module, the member module, the coach module, and the administrator module. In addition, some highly scalable codes will be reused in modules, e.g. "Email sending module" (show in Figure 4).

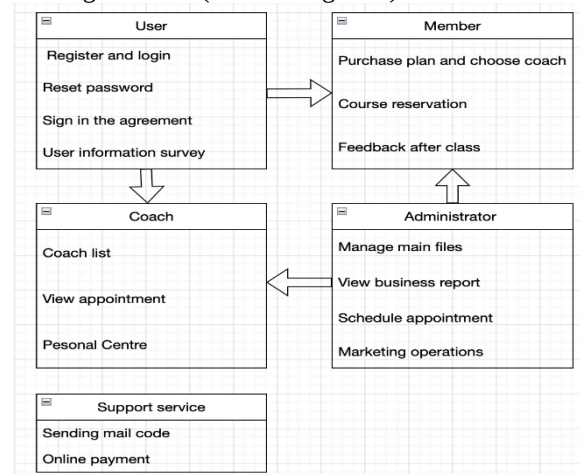


Figure 4. The Overall Module Design

For the justification, the function cohesion in each module is high, the logic between business is closely connected, and the dependency between different role modules is low.

3.2 High-level Process Flow

Our system has successfully developed the necessary functions and some exquisite designs for a fitness reservation system. The different modules in the system have clear division of labor and tight connections (The flow chart could be seen in the Appendix 2).

In our system, the user initially enters the fitness reservation main webpage and selects user registration. After successful registration, they can log in. If the password is incorrect, you can still retrieve and reset the password. It is worth mentioning that our system has also developed functions to prevent duplicate login and suggest changing passwords after a long period of non login. After successful login, the system will automatically recognize the personnel category and redirect to different pages. If a member successfully logs in, they will be redirected to the main page. After entering the user's personal information, they can purchase a fitness plan. After successful payment, they can choose a specific coach and choose a time. After the course ends, members can view the appointment history to provide feedback. If an administrator logs in, they will see the administrator page to manage appointments, view business reports, and manage files.

3.3 Software Support Services

1. Database related services:

• MySQL

In the domain of database development, our team chooses MySQL. It saves data in different tables, improving operation speed and flexibility. The connection among diverse tables is realized through constraints, thereby safeguarding data precision, integrity, and cohesion.

• MyBatisPlus

MyBatisPlus [5] is a powerful tool for dealing with database data interaction in modern web applications, providing workers with more efficient and flexible database operations. It facilitates the writing and maintenance of database while lowering error rates and rework.

2. Webpage navigation related services:

Navigation elements predominantly consist of responsive buttons, enhancing user convenience. The homepage features an integrated sidebar navigation, complemented by multi-tiered buttons offering expanded functionalit. This exhibits scalability and reusability, facilitating adaptation to evolving web functionalities and accommodating diverse user roles.

3. Other support services:

• SpringBoot mail

This project uses Springboot to integrate Mail to realize mail sending, primarily utilized for user registration and preemptive notification of plan expiration. The former sends a verification code to the user, while the latter proactively communicates with the user before the plan expires (see Figure 5).

```
public void sendSimpleMail(MailRequest mailRequest) {
    SimpleMailMessage message = new SimpleMailMessage();
    checkMail(mailRequest);
    //The sender of the email
    message.setFrom(sendMail);
    //1 or more recipients
    message.setTo(mailRequest.getSendTo().split(" "));
    //subject of an email
    message.setSubject(mailRequest.getSubject());
    //mail content
    message.setText(mailRequest.getText());
    //post time
    message.setSentDate(new Date());

    javaMailSender.send(message);
    logger.info("Send email successfully:{}->{}",sendMail,mailRequest.getSendTo());
}
```

Figure 5. SpringBoot mail Service

•Git

Git serves as a distributed version control system which has ability to efficiently manage projects. Throughout the development lifecycle, team members establish individual warehouse, facilitating timely updates and code synchronization for incremental development.

•Maven

Maven, a project management, and automated build tool, simplifies the management of external dependencies. It mitigates issues such as dependency miscellaneous, version inconsistencies, and conflicts. Additionally, Maven provides a standardized project structure, facilitating collaboration among developers.

3.4 Coding Structure and Convention

1. Code Directory Structure:

According to the SpringBoot framework, the functionality of each directory is specified in the developer specification document before the software development (see document in Appendix 1).

2. Method-calling Conventions:

To reduce the coupling between components, the controller calls the service method through the interface, that is, each service interface corresponds to one or more implementation classes.

3. Naming Notations:

- Use CamelCase: for all method and variable names. In CamelCase, the first letter of each word is capitalized except for the first word, and there are no spaces between words.
- Use PascalCase for all class names. PascalCase is like CamelCase, but the first letter of each word is also capitalized.
- Use UPPER_CASE for all constants, e.g. 'MAX_SIZE'
- All names should have practical meaning in the real world. Object plus behavior naming is recommended.
- Interface implementation classes should always be named as the interface name plus "impl".

4. Code Maintenance Conventions:

Keep easy-to-understand annotations up to date as you develop and remove useless code and comments before uploading to git.

5. **TODO List** : Incomplete development service module, submit code please mark "TODO". Add the developer's name and date to avoid further omission (see Figure 6).

```
*
* @author Gu Qibin
* @since 2024-04-14
* TODO
```

Figure 6. The TODO Annotation in the Development.

3.5 Software Configuration and Production Environment

• Software Configuration:

For frontend, we use HTML, CSS, and JavaScript to create a user interface for customers, managers, and coaches and Thymeleaf templates are utilized as the engine to provide users with a satisfied user experience from the client side to the server side. As for the backend, the system is configured using the SpringBoot framework and the MVC architecture, enabling flexible changes to the system and helping the system maintain more modular and scalable to meet rapid changing requirements. When it comes to the database, we pick MySQL as our preferred database management system so that all our data can be stored and managed efficiently.

• Production Environment:

System	Ubuntu 22.04 64x
Processor	Intel ® Xeon ® Platinum Scalable Processors, 2-core(vCPU), 4 GiB
Public network	bandwidth 2 Mbps
Java OpenJDK	version 19.0.2
MySQL	Version 8.0.36

Our system is deployed on the **AliyunServer** to maintain the system to function correctly.

IV. Software testing

1. Unit test

In the development phase of our fitness appointment booking system, unit testing is critical to verify the functionality and reliability of individual components. It ensures that each function within our system behaves as expected under various conditions, thereby enhancing the robustness and accuracy of our application. One specific scenario we focused on was the **makeAppointment** method, which handles user requests to schedule new fitness sessions.

Function Description: The objective of this function is to allow users to schedule new appointments through the system. It validates user input, ensures that appointment times are logical (e.g., start time is before end time, and the date is not in the past), and stores the appointment data if all conditions are met.

Example Test Case:

•Test Case ID:

testMakeAppointmentWithInvalidDate

•Test code snippet Excerpts:

```
1 @Test
2 public void testMakeAppointmentWithPastDate()
3     throws Exception {
4     // Scenario: Appointment date is in the
5     // past
6     String pastDate = LocalDate.now().minusDays
7     (1).toString(); // Yesterday's date: May
8     // 5th, 2024
9     String requestBody = String.format("""
10     {
11         "date": "%s",
12         "startTime": "10:00",
13         "endTime": "11:00",
14         "trainerId": "1",
15         "memberId": "123"
16     }
17     """, pastDate);
18
19     MvcResult result = mockMvc.perform(post("/
20     FitnessAppointment_System/api/
21     makeAppointment")
22         .contentType(MediaType.
23         APPLICATION_JSON)
24         .content(requestBody))
25         .andReturn();
26
27     // Assert that the status code is 400 (Bad
28     // Request)
29     assertEquals(HttpStatus.BAD_REQUEST.value(),
30         result.getResponse().getStatus());
31
32     logger.info("Test for past date appointment
33     : {} ", result.getResponse().
34     getContentAsString());
35 }
```

•Test Result:

Test Case	Input	Expected Output	Actual Output	Result
Past Date Appointment	date: May 5th, 2024, time: 10:00-11:00	Status: 400, Message: Invalid Date	Status: 400, Message: Invalid Date	Pass
End Time Before Start Time	date: May 7th, 2024, time: 11:00-10:00	Status: 400, Message: Invalid Time Order	Status: 400, Message: Invalid Time Order	Pass
Valid Appointment	date: May 8th, 2024, time: 14:00-15:00	Status: 200, Message: Success	Status: 200, Message: Success	Pass

Figure 7. Test Result of Example Unit Test

We used JUnit and Mockito frameworks for unit testing during the design of this piece [6]. JUnit is used to define test structures and execute tests, while Mockito is used to simulate dependent services, ensuring the independence and repeatability of each test. Not only that, but we also use Spring Boot's MockMvc to simulate HTTP requests, to simulate front-end and back-end fetch requests, which is an ideal way to validate the controller layer as we discussed, because it simulates MVC behavior directly without having to start the full HTTP service. The conclusion of the unit tests is that the functionality of the software in our system works properly.

2. Integration Testing

Integration testing involves testing the interactions between different components or modules of the

system to ensure that they work together as expected. In the context of the online appointment system, integration testing would focus on verifying the interactions between the frontend and backend systems, as well as between various backend components.

Take the integration test of retrieving password through email verification code as an example, the function is designed to integrate front-end to back-end, integrate login module to email sending module and integrate database.

Expected Test Steps:

- **Step1:** Simulate a user request to retrieve a forgotten password from the frontend.
- **Step2:** Frontend sends a request to the backend API endpoint responsible for handling password retrieval requests.
- **Step3:** The login module retrieves the user's email address record from the database and sends it to the email sending module's interface.
- **Step4:** The email sending module sends a preset email and returns a random verification code to the login module.

Test Results:

As expected, the front-end interaction to retrieve passwords, the back-end interface calls, and the database connection work fine. As shown in the Figure 8, the test mailbox received an email from the server with a randomly generated verification code. Therefore, this integration test passes.



Figure 8. The Successful Result of Verification Code Sending.

3. Acceptance Testing

Acceptance testing involves verifying that the system meets the requirements and expectations of stakeholders. It focuses on testing the system's functionality from an end-user perspective, including user stories and scenarios, ensuring that it behaves as intended and meets the specified acceptance criteria. This section follows with an example of the PBI ABS#002 acceptance test for business report generation.

Test Case ID: test PBI ABS#002

Scenarios	Test inputs	Expected Output	Actual Output	Result
Scenario1	Select correct range of beginning and ending date	Display correct result of selected transactions with volume and transaction amount	Actual result displayed	Pass
Scenario2	Select incorrect range of date	Display error message "Invalid setting, please check it!" in red	Error message displayed	Pass
Scenario3	Press "generate" button with expected time range	Generate two figures of statistical information (order volume and total revenue)	Figures generated as expected	Pass

In addition, for the UI requirement, the acceptance test ensure that the UI is intuitive, responsive, and visually consistent (show in the Figure 9).

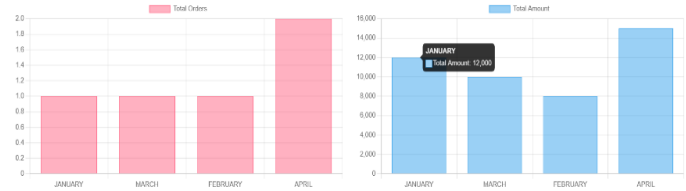


Figure 9. The User-friendly UI of the Visual Chart.

V. Conclusion and future work

In conclusion, our group has successfully designed and implemented a web-based Fitness Appointment Management System to address the challenges faced by fitness centers and customers alike. We adopted the MVC architecture to ensure clear structure, code reusability, and separation of concerns. The system caters to four main user types: visitors, members, coaches, and managers, providing tailored functionalities to enhance the fitness appointment experience. Throughout the project, we achieved several milestones, including the development of a user-centric platform allowing members to register, purchase fitness plans, and schedule appointments. Additionally, we created a management background for managers to oversee member and coach information, visualize data, and send marketing notifications. The integrated system passed the tests and was released as version 1.0 (see Appendix 3 for PBI details).

However, we encountered challenges during the development process, particularly related to version control development, ensuring data security, and optimizing system performance. These challenges highlighted the need for continuous improvement and refinement in our approach.

Looking ahead, our group plans to continuously monitor user feedback and iteratively improve the system based on user needs and preferences. We aim to enhance system security measures to protect user data and prevent unauthorized access in the long-term support for the system.

Reference

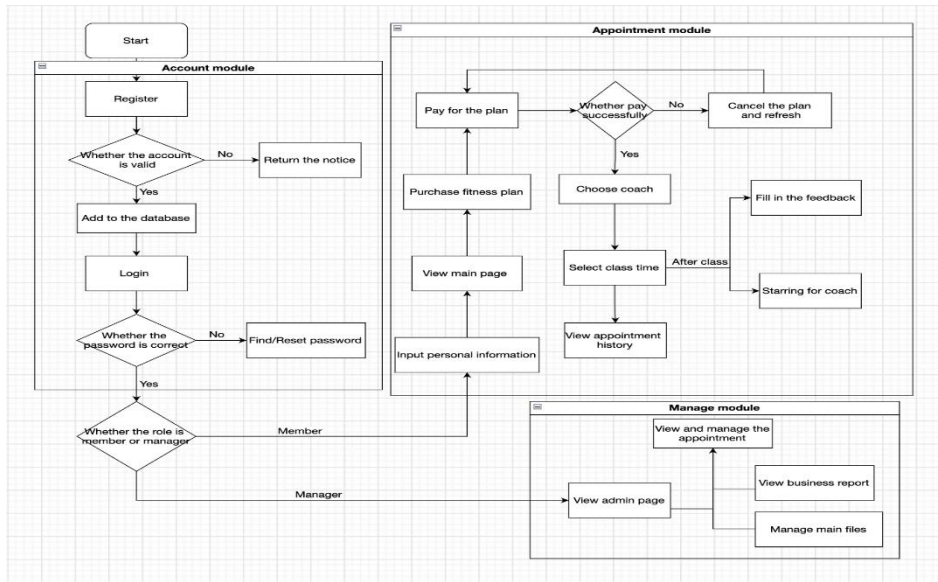
- [1] T. J. X. P. t. n. Reenskaug, "Models-views-controllers," 1979.
- [2] A. Singh, P. Chawla, K. Singh, A. J. n. I. C. o. T. i. E. Kumar Singh, and Informatics, "Formulating an MVC Framework for Web Development in JAVA," pp. 926-929, 2018.
- [3] D. Setiawan, Buku sakti pemrograman web: html, css, php, mysql & javascript. Anak Hebat Indonesia, 2017.
- [4] M. J. I. J. o. D. M. S. Demba, "Algorithm for Relational Database Normalization Up to 3NF," vol. 5, pp. 39-51, 2013.
- [5] D. Zhang, Z. Wei, and Y. Yang, "Research on lightweight MVC framework based on spring MVC and mybatis," in 2013 sixth international symposium on computational intelligence and design, 2013, vol. 1, pp. 350-353: IEEE.
- [6] S. Acharya, Mastering unit testing using Mockito and JUnit. Packt Publishing Ltd, 2014.

Appendix

Appendix 1: Specification documents

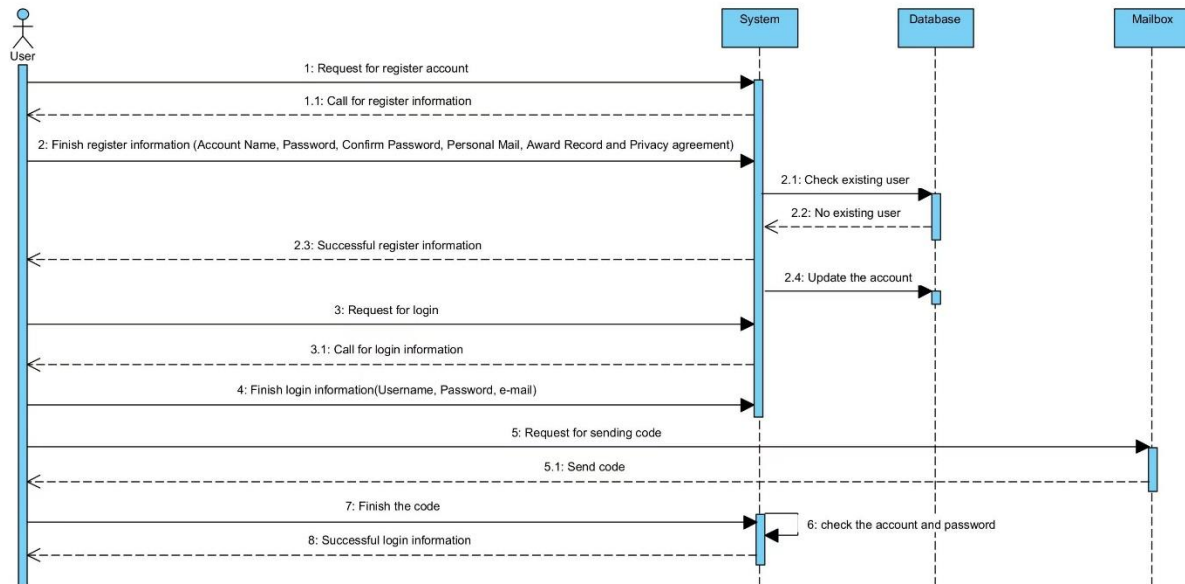
```
|---/src/main/com.cpt202.group23.Fitness_Appointment_System
|---|---|---/config: Configuration class storage location
|---|---|---/controller : control layer
|---|---|---/service: Business code layer
|---|---|---/entity: storing database entity classes
|---|---|---/mapper: Interaction layer with database
|---|---|---/src/main/resource: Stores resource files, such as static resource files, configuration files, page files, etc.
|---|---|---/src/main/resource/mapper: an XML file that stores the MyBatis operation database
|---|---|---/src/main/resource/static: Stores static resources such as CSS styles, images, etc
|---|---|---/src/main/resources/templates: Store template files such as thymeleaf template files
|---|---|---|
|---/src/test unit test class Java file
```

Appendix 2: Flow chart

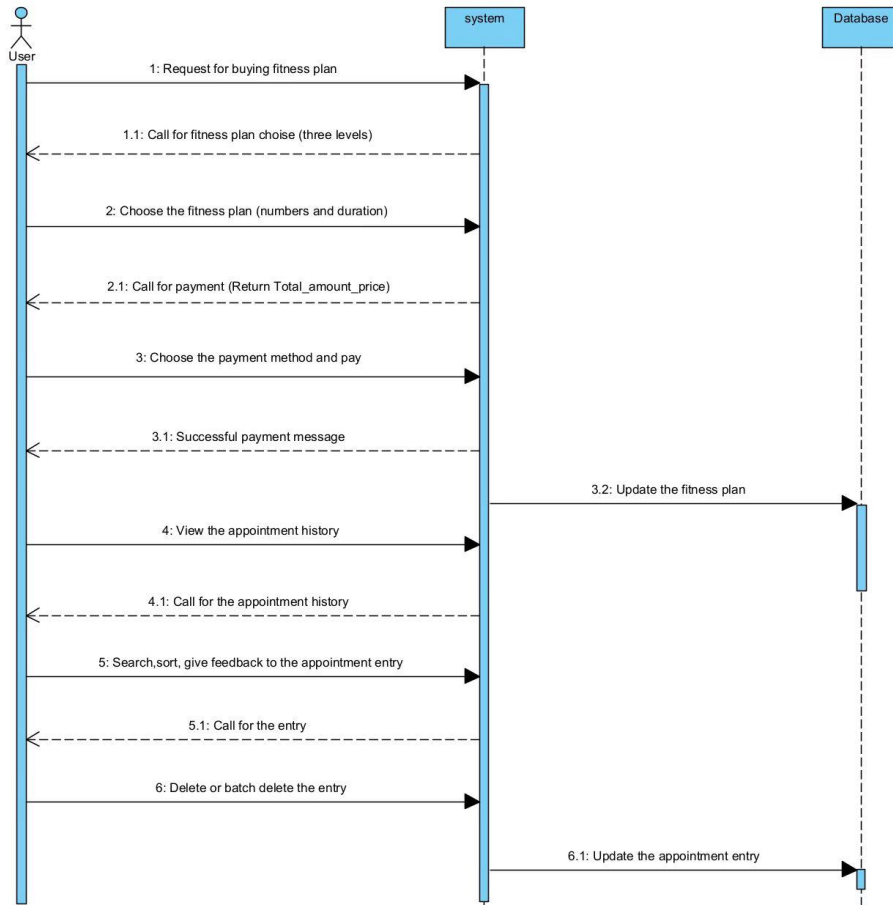


Appendix 3: Sequence diagram

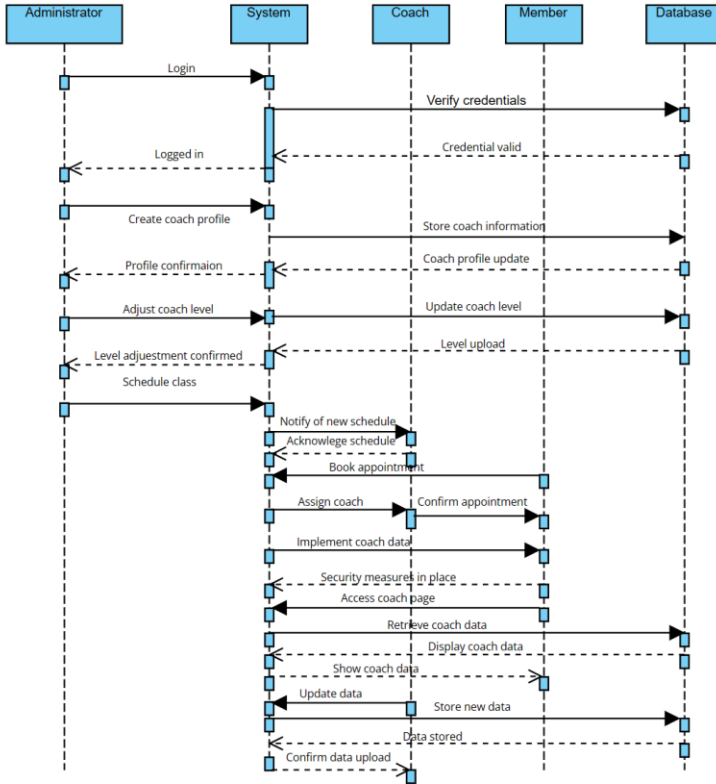
(a) Register and login



(b) Appointment



(c) Coach



Appendix 4: Sprint

Sprint Progress Report						
Project Name	Fitness_Appointment_System					
Team Number	8					
Team Name	Group23					
Team Leader	Qibin Gu					
TA	Yang Liu					
Week #	Sprint #	PBI	Owner	Start Date	Expected Completion Date	Actual Completion Date
1		1Trainer Manager's Maintenance Page for Adding Trainer Information	Qibin Gu	3/25/2024	3/26/2024	3/26/2024
		1Trainer Manager's Maintenance Page for Modifying and Deletion	Qibin Gu	3/25/2024	3/26/2024	3/26/2024
		1The Record for the Manager's Operation on Trainer Table	Qibin Gu	3/27/2024	3/28/2024	3/28/2024
		1Manage Customer information for Additions and Modification	Jixiao He	3/25/2024	3/26/2024	3/26/2024
		1Manage Customer information for Deletion	Jixiao He	3/27/2024	3/27/2024	3/27/2024
		1Basical Construction of Website Home Page	Zhongheng Liu	3/25/2024	3/26/2024	3/26/2024
		1The front-end of Login Page	Zhongheng Liu	3/27/2024	3/28/2024	3/28/2024
		1Identity Selection for User Login Page	Zhongheng Liu	3/29/2024	3/30/2024	3/30/2024
		1Appointment History record table	Yucheng Gu	3/25/2024	3/26/2024	3/26/2024
		1Show Appointment History	Yucheng Gu	3/27/2024	3/28/2024	3/28/2024
		1Fitness Trainer Level Rating View	Jiarui Chen	3/27/2024	3/28/2024	3/28/2024
		1Fitness Trainer Information View	Jiarui Chen	3/29/2024	3/30/2024	3/30/2024
		1Fitness Plan Table	Hanling Zheng	3/25/2024	3/26/2024	3/26/2024
		1Browsing and selecting fitness plan	Hanling Zheng	3/26/2024	3/27/2024	3/27/2024
		1Database of register login information	Haoran Qin	3/26/2024	3/27/2024	3/27/2024
		1Registration UI	Haoran Qin	3/28/2024	3/29/2024	3/29/2024
		1Reset Password Front-end	Haoran Qin	3/30/2024	3/31/2024	3/31/2024
		1the Tabbar of User Dashboard	Yinuo Wang	3/25/2024	3/27/2024	3/27/2024
		2View and Schedule Appointment Calendar	Qibin Gu	4/5/2024	4/7/2024	4/6/2024
		2Member Group Analysis Report	jixiao He	4/3/2024	4/4/2024	4/4/2024
		2Coach Dashboard	Yinuo Wang	4/3/2024	4/4/2024	4/4/2024
		2Coach's personal center page	Yinuo Wang	4/5/2024	4/6/2024	4/6/2024
		2Appropriate Level of Fitness Coach Assignment for Appointment	Jiarui Chen	4/3/2024	4/5/2024	4/5/2024
2		2Making the Appointment by Users	Yucheng Gu	4/4/2024	4/5/2024	4/6/2024
		2Send Email Function	Zhongheng Liu	4/3/2024	4/4/2024	4/4/2024
		2Users Login	Zhongheng Liu	4/5/2024	4/7/2024	4/7/2024
		2Protocol Signing Interface Design	Haoran Qin	4/3/2024	4/5/2024	4/5/2024
		2Back-end development of registration system	Haoran Qin	4/6/2024	4/7/2024	4/7/2024
		2Purchase a Fitness Plan	Hanling Zheng	4/4/2024	4/6/2024	4/6/2024

3	3Send Marketing Notifications	Qibin Gu	4/12/2024	4/14/2024	4/14/2024
	3Monthly user plan statistics table	jixiao He	4/8/2024	4/10/2024	4/10/2024
	3Coach Promotions/Demotions	Jiarui Chen	4/10/2024	4/12/2024	4/12/2024
	3Delete the Appointment by Users	Yucheng Gu	4/11/2024	4/12/2024	4/12/2024
	3Coach Account Registration	Zhongheng Liu	4/9/2024	4/10/2024	4/10/2024
	3Login permissions for the coach account	Zhongheng Liu	4/10/2024	4/12/2024	4/12/2024
	3Sign in the Privacy Agreement	Haoran Qin	4/10/2024	4/12/2024	4/12/2024
	3Record management of fitness plan	Hanling Zheng	4/11/2024	4/13/2024	4/12/2024
	4Business Report of Specific Duration Query	Qibin Gu	4/17/2024	4/19/2024	4/19/2024
	4Chart Generation in the Business Report	Qibin Gu	4/18/2024	4/19/2024	4/19/2024
4	4Trainer Business Report	Jixiao He	4/17/2024	4/19/2024	4/19/2024
	4Class Scheduling for Coach	Jiarui Chen	4/18/2024	4/19/2024	4/19/2024
	4Sort the Appointment Record by Users	Yucheng Gu	4/11/2024	4/12/2024	4/12/2024
	4Limitation on User's Repeated Logins	Zhongheng Liu	4/16/2024	4/17/2024	4/17/2024
	4Fitness Plan Status Updates	Yinuo Wang	4/20/2024	4/21/2024	4/21/2024
	4User Profile	Haoran Qin	4/19/2024	4/20/2024	4/20/2024
	4User Account Recovery	Haoran Qin	4/17/2024	4/19/2024	4/19/2024
	4Renewal Process of Expired Fitness Plan	Hanling Zheng	4/18/2024	4/20/2024	4/20/2024
	5Integration Test on Different modules	Qibin Gu	4/25/2024	4/29/2024	4/29/2024
	5Admin Dashboard	Jixiao He	4/24/2024	4/25/2024	4/25/2024
5	5Feedback and Comments on Appointment	Yucheng Gu	4/24/2024	4/26/2024	4/26/2024
	5Server Deployment	Yinuo Wang	4/23/2024	4/24/2024	4/24/2024
	5Database Deployment	Yinuo Wang	4/25/2024	4/25/2024	4/25/2024
	5Member Physical Situation Submission	Haoran Qin	4/26/2024	4/28/2024	4/28/2024
	5View Status of Current Appointment	Hanling Zheng	4/25/2024	4/27/2024	4/27/2024
	6	Qibin Gu			
	6	Hanling Zheng			
	6Acceptance Testing	Yinuo Wang	5/1/2024	5/5/2024	5/6/2024
	6First Software Deployment	Yinuo Wang	4/30/2024	4/30/2024	4/30/2024
	6User interface Improvements	Haoran Qin	4/29/2024	4/30/2024	4/30/2024
6	6Debug and Improvements for testing	whole group	5/7/2024	5/10/2024	5/10/2024
	7Second Software Deployment	Yinuo Wang	5/11/2024	5/11/2024	5/11/2024

Appendix 5.
 Group 23 database details
 (All contents of the database are named with lowercase letters.)
 The name of the unified database is fitness_system
 coach,member,users,fitness_plan, appointment, order,

教练表（coach） by Qibin

coach 输入一个 SQL 表达式来过滤结果 (使用 Ctrl+Space)						
	123 id	abc name	abc contact	123 level	123 salary	123 experience
1	1	John Doe	john.doe@example.com	3	6,000	5
2	7	Mike Smith	mike.smith@example.com	4	8,000	3

列名	#	数据类型	非空	自增	键	默认	额外的	表达式	注释
123 id	1	int	[v]	[v]	PRI		auto_incre...		
abc name	2	varchar(100)	[v]	[]					
abc contact	3	varchar(100)	[v]	[]					
123 level	4	int	[v]	[]					only could ...
123 salary	5	decimal(10,...	[v]	[]					
123 experience	6	int	[v]	[]		0			The numbe...

field	type	NULL	INCRE	KEY	Function	description
id	int	Y	Y	pri	coach id	
name	varchar	N	N	/	name	
contact	varchar	N	N	/	Storage Contact Information	The default is email

level	int	N	N	/	Coach star rating	The integer types used (3 for 3star,4 for 4star, and 5 for 5star) can only be 3,4,5. Otherwise, reject the change.
salary	decimal(10, 2)	N	N	/	salary	two decimal places
experience	int	N	N	/	Work experience (min. Years))	Starting at 0, indicating years of work, is used for star ratings
user_id	int	Y	N	foreign key	referencing user table	check account

```
-- fitness_system.coach definition
```

```
CREATE TABLE `coach` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
  `contact` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
  `level` int DEFAULT NULL COMMENT 'only could be 3 or 4 or 5',
  `salary` decimal(10,2) DEFAULT NULL,
  `experience` int DEFAULT '0' COMMENT 'The number of years used to record teaching experience is counted as integers only',
  `user_id` int NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

课程预约表（appointment） by Qibin

field	type	NULL	INCRE	KEY	Function	description
appo_id	int	Y	Y	PRI	id	
date	date(YYYY:MM:DD)	Y	N		Record the date of the course appointment	
start_time	time (HH:mm:ss)	Y	N		The start time of the course	
end_time	time (HH:mm:ss)	Y	N		The end time of the course	
trainer_id	int	Y	N	Foreign key(coach table)	The responsible instructor	In addition, key deletion and

					of the course	update are cascaded
member_id	int	Y	N	Foreign(member id)	Participants in the course	In addition, key deletion and update are cascaded

列名	#	数据类型	非空	自增	键	默认	额外的	表过
123 appo_id	1	int	[v]	[v]	PRI		auto_incre...	
🕒 date	2	date	[v]	[]				
🕒 start_time	3	time	[v]	[]				
🕒 end_time	4	time	[v]	[]				
123 trainer_id	5	int	[v]	[]	MUL			
123 member_...	6	int	[v]	[]	MUL			

tment_id	🕒 date	🕒 start_time	🕒 end_time	123 trainer_id	123 member_id
1	2024-04-14	08:00:00	09:00:00	1 ↗	1 ↗
3	2024-04-16	14:07:00	15:07:00	1 ↗	1 ↗
4	2024-04-15	15:07:00	17:07:00	3 ↗	1 ↗
5	2024-04-15	15:07:00	17:07:00	1 ↗	1 ↗
6	2024-04-15	07:22:00	09:28:00	1 ↗	1 ↗
7	2024-04-17	17:33:00	20:33:00	1 ↗	1 ↗

```

CREATE TABLE `appointment` (
  `appointment_id` int NOT NULL AUTO_INCREMENT,
  `date` date NOT NULL COMMENT 'the actual date of this appointment',
  `start_time` time NOT NULL COMMENT 'the start time of the appointment',
  `end_time` time NOT NULL COMMENT 'the end time of this appointment',
  `trainer_id` int NOT NULL COMMENT 'the trainer who work for this appointment',
  `member_id` int NOT NULL COMMENT 'the member id who own this appointment',
  PRIMARY KEY (`appointment_id`),
  KEY `fk_trainer_id` (`trainer_id`),
  KEY `fk_member_id` (`member_id`),
  CONSTRAINT `fk_member_id` FOREIGN KEY (`member_id`) REFERENCES `member` (`id`)
  ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_trainer_id` FOREIGN KEY (`trainer_id`) REFERENCES `coach` (`id`) ON
  DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

计划订单表 (order) by Qibin (注: 此表应该有外键但是下方的 SQL 没有添加)

field	type	NULL	INCRE	KEY	Function	description
order_id	int	Y	Y	PRI	id	order reference
member_id	int	Y	N	Foreign(member 中的 id)	Record the id of the purchaser	Foreign keys associate student information
amount	decimal(10,2)	Y	N		payment amount	two decimal places
pay_method	varchar	Y	N		payment method	"Alipay" or "Wechatpay"
start_time	date	Y	N		The effective	If there is no plan before the

					time of the plan	purchase, it is the current time; if the plan has not expired, it is the original start time unchanged
expiration_date	date	Y	N		The expiration time of the plan	If it is renewed, the field data is extended accordingly
status	enum("processing","completed")	Y	N		The status of the plan	Two types, one completed and one in progress, can be used to identify whether the user still has plans.
plan_id	int	Y	N	Foreign (plan 中主键"id")	To bind the purchase plan level	1:sliver 2:gold 3:diamond

列名	#	数据类型	非空	自增	键	默认	额外的 auto_incre...	表达式	注释
123 order_id	1	int	[v]	[v]	PRI				
123 member_...	2	int	[v]	[]					
123 amount	3	decimal(10,...	[v]	[]					
abc pay_meth...	4	varchar(100)	[v]	[]					alipay or w...
start_time	5	date	[v]	[]					
expiration...	6	date	[v]	[]					
abc status	7	enum('proc...	[v]	[]					
123 plan_id	8	INT	[v]	[]					

```
-- fitness_system.`order` definition
```

```
CREATE TABLE `order` (  
  `order_id` int NOT NULL AUTO_INCREMENT,  
  `member_id` int NOT NULL,  
  `amount` decimal(10,2) NOT NULL,  
  `pay_method` varchar(100) NOT NULL COMMENT 'alipay or wechat pay',  
  `start_time` date NOT NULL,  
  `expiration_date` date NOT NULL,  
  `status` enum('processing','completed') NOT NULL,  
  `plan_id` int NOT NULL,  
  PRIMARY KEY (`order_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

用户表 (users) by Haoran

users

列 7

- user_id int (auto increment)
- name varchar(30)
- email varchar(100)
- password varchar(50)
- login_status tinyint(1) = 0
- code int
- status_code int

field	type	NULL	INCRE	KEY	Function	description
user_id	int	Y	Y	pri	id	

name	varchar	Y	N	/		
email	varchar	Y	N	/	Used to receive a CAPTCHA	
password	varchar	Y	N	/		
login_status	boolean	Y	N	/	Online or not	0 is not online and 1 is online
code	int	Y	N	/	verification code	random generated
status_code	int	Y	N	/	identification system	1customer 2coach 3manager

```

create table users
(
    user_id    int auto_increment
              primary key,
    name       varchar(30)    not null comment 'not same',
    email      varchar(100)   not null comment 'be used to send verification code ',
    password   varchar(50)    not null comment 'accept numbers and letters',
    login_status tinyint(1) default 0 not null comment '0 represent offline,1 represent online',
    code       int           null comment 'the element of mail setting',
    status_code int          not null comment '1 = customer, 2 = coach,3 = manager'
);

```

后台端会员表 (member) by Jixiao

field	type	NULL	INCRE	KEY	Function	description
id	int	N	Y	pri	id	

name	varchar	Y	N	/	member name	/
gender	varchar		N	/	gender	F/M
age	int	Y	N	/		
contact	varchar	Y	N	/	contact information	mail address
description	int		N	/	Health information	/
user_id	int	N	N	foreign key	referencing user table	check account

```

DROP TABLE IF EXISTS `member`;
CREATE TABLE `member` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `contact` varchar(255) DEFAULT NULL,
  `gender` varchar(10) DEFAULT NULL,
  `age` int(11) DEFAULT NULL,
  `name` varchar(255) DEFAULT NULL,
  `description` varchar(255) DEFAULT NULL,
  `user_id` int(11) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=18 DEFAULT CHARSET=latin1;

```

健身计划表 (FitnessPlan) by Hanlin and Yuchen

field	type	NULL	INCRE	KEY	Function	description
id	int	Y	Y	primary	id	
name	string	Y	N	/	the title of plans	
price	double	Y	N	/	price	
duration	integer	Y	N	/	plan duration	
description	text	N	N	/		

```

CREATE TABLE `FitnessPlan` (
  `id` INT NOT NULL,
  `name` varchar(255) NOT NULL,

```

```

`price` double NOT NULL,
`duration` int NOT NULL COMMENT '以月为单位', -- "以月为单位" translates to "by month"
`description` text,
PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Appendix 6.

Individual Contribution:

CPT202 Assignment 2

Individual Contribution for Group Report and Presentation

Group Number:

Name	ID Number	Contribution (%)
1.Qibin Gu	2143132	12.5%
2.Zongheng Liu	2144695	12.5%
3.Haoran Qin	2143434	12.5%
4.Jiarui Chen	2145016	12.5%
5.Yinuo Wang	2143466	12.5%
6.Jixiao He	2144984	12.5%
7.Yuchen Gu	2142038	12.5%
8.Hanlin Zheng	2145799	12.5%
9.		

Signed by all members:

顾齐斌 顾立辰 陈佳睿
 刘宗衡 秦浩然 郑博
 何骁霄 王一诺